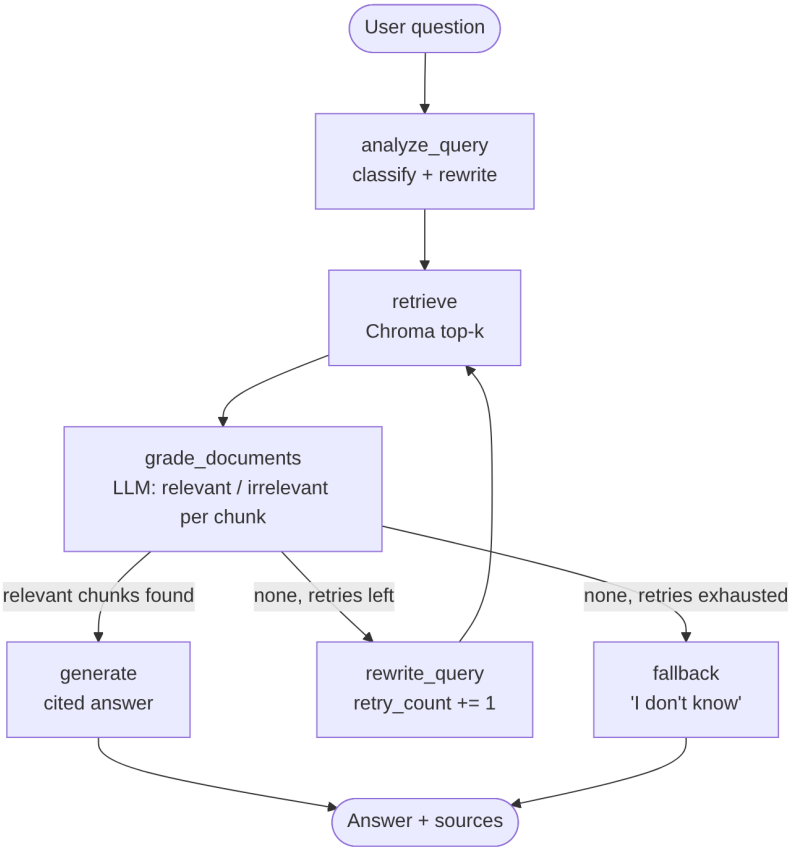
RAG Technical Documentation Assistant

A Retrieval-Augmented Generation service that answers questions about technical documentation (the official LangGraph docs by default). The pipeline is a **self-corrective LangGraph workflow**: it rewrites the query, retrieves chunks, has an LLM grade each chunk for relevance, retries with a new query if nothing relevant comes back, and only then generates a cited answer. If retries run out it says "I don't know" instead of hallucinating. It is served through FastAPI.

Stack: Python, LangGraph, FastAPI, ChromaDB, OpenAI (`gpt-4o-mini` + `text-embedding-3-small`).

Architecture



Node	Responsibility
`analyze_query`	LLM (structured output) classifies the question (`conceptual` , `how-to` , `troubleshooting` , `api-reference`) and rewrites it into a retrieval-friendly query. The type also tunes the answer style.
`retrieve`	Similarity search in ChromaDB, top-k chunks with source metadata.
`grade_documents`	One LLM call per chunk (run in parallel) returns `relevant` / `irrelevant` . Irrelevant chunks are dropped.
`rewrite_query`	Writes a differently-worded query and increments `retry_count` .
`generate`	Answers only from the surviving chunks with `[n]` citations.
`fallback`	Returns a polite "not found in the docs" message.

Conditional edge (`app/graph/edges.py`): after grading, any relevant chunk goes to `generate` ; none and `retry_count < MAX_RETRIES` goes to `rewrite_query` ; none and retries exhausted goes to `fallback` .

State schema (`app/graph/state.py`)

Field	Purpose
`question`	Original user text, **never overwritten**. Grading and generation use it so a bad rewrite cannot drift the answer.
`rewritten_query`	What retrieval actually searches with; set by `analyze_query` and `rewrite_query` .
`query_type`	Classification, used to pick the answer style.
`documents`	Raw retrieval output.
`relevant_documents`	Chunks that survived grading (kept separate so grading is observable/testable).
`retry_count`	Incremented only in `rewrite_query` ; the router compares it to `MAX_RETRIES` , which guarantees the loop terminates.
`answer` , `sources`	Final output. `sources` only lists chunks actually cited as `[n]` .
`fallback_used`	Lets the API and evaluation distinguish a real answer from "I don't know".

Project layout

```
app/
  main.py            FastAPI endpoints
  config.py          all tunables (env-overridable)
  llm.py             LLM + embedding factories (lazy)
  vectorstore.py     ChromaDB access, document listing
  ingestion.py       load -> convert -> chunk -> embed -> store
  storage.py         SQLite for queries + feedback
  schemas.py         request/response models
  graph/             state, nodes, edges, builder
scripts/
  fetch_corpus.py    download the corpus into data/docs/
  ingest.py          index data/docs/ into ChromaDB
  run_eval.py        run sample questions and show routing
data/
  docs/              the corpus
  test_questions.json
tests/               pytest suite (no API key or network needed)
```

Setup

```
python -m venv .venv
source .venv/bin/activate            # Windows: .venv\Scripts\activate
pip install -r requirements.txt

cp .env.example .env                 # then put your OpenAI key in .env
```

Run

```
python scripts/fetch_corpus.py       # downloads 4 LangGraph doc pages into data/docs/
python scripts/ingest.py             # chunk + embed + store in ./chroma_db
uvicorn app.main:app --reload        # API at http://127.0.0.1:8000  (docs at /docs)
```

Optional sanity check of routing (answered vs fallback):

```
python scripts/run_eval.py
```

Run the tests (they mock the LLM, so no key is needed):

```
pytest
```

Corpus

Four pages from the official LangGraph documentation: *Overview*, *Graph API* (state, nodes, edges), *Persistence* (checkpointers) and *Agentic RAG*. They are fetched from URLs by `scripts/fetch_corpus.py`. You can also add any Markdown/text/HTML file or URL at runtime with `POST /ingest`.

API

`POST /query`

```
curl -X POST http://127.0.0.1:8000/query \
  -H "Content-Type: application/json" \
  -d '{"question": "How do I add a conditional edge to a StateGraph?"}'
```

Response shape:

```
{
  "query_id": "3f0c...",
  "question": "How do I add a conditional edge to a StateGraph?",
  "answer": "... with citations like [1] ...",
  "sources": [
    {"id": 1, "source": "langgraph_graph_api.md", "title": "...",
     "section": "Graph API > Conditional edges", "snippet": "..."}
  ],
  "query_type": "how-to",
  "rewritten_query": "...",
  "retries": 0,
  "fallback_used": false
}
```

An out-of-scope question returns `"fallback_used": true`, `"sources": []` and `"retries": 2`.

Status codes: `200` ok, `422` invalid input, `409` index is empty, `502` LLM provider error, `500` unexpected error.

`POST /ingest`

```
# upload files
curl -X POST http://127.0.0.1:8000/ingest -F "files=@my_notes.md" -F
"files=@page.html"

# or URLs (comma or newline separated)
curl -X POST http://127.0.0.1:8000/ingest -F
"urls=https://docs.langchain.com/oss/python/langgraph/quickstart"
```

Returns `{"documents_ingested": 1, "chunks_added": 12, "sources": [...]}`. Re-ingesting the same source replaces its chunks (no duplicates). `400` for unsupported file types / bad URLs, `413` for files over 2 MB.

`GET /documents`

```
curl http://127.0.0.1:8000/documents
```

Returns each indexed source with its chunk count, plus totals.

`POST /feedback`

```
curl -X POST http://127.0.0.1:8000/feedback \
  -H "Content-Type: application/json" \
  -d '{"query_id": "<id from /query>", "rating": "up", "comment": "clear answer"}'
```

`rating` must be `up` or `down`. Returns `201`; `404` if the `query_id` is unknown. Feedback is stored in SQLite (`app.db`).

Design decisions and tradeoffs

Chunking. Technical docs mix prose and code, so naive fixed-size splitting hurts. Strategy: (1) convert HTML to Markdown so headings and code fences are preserved; (2) split on `#` / `##` / `###` headers first so a chunk never spans two sections; (3) split oversized sections with a Markdown-aware recursive splitter at **800 characters / 100 overlap**, which prefers paragraph and code-fence boundaries; (4) prefix every chunk with `[Page title > Section]`. 800 chars is large enough to keep a code example next to its explanation but small enough that retrieval stays precise. The context prefix means a chunk like "return {...}" still embeds with the section it belongs to. Tradeoff: the prefix slightly inflates tokens, and a very long code block can still be split.

Embeddings and vector store. `text-embedding-3-small` is cheap and strong for English technical text. ChromaDB persists locally with no server, which keeps setup simple. Chunk IDs are deterministic hashes and a source's old chunks are deleted before re-ingest, so ingestion is idempotent.

Grading against the original question. The grader sees the user's original `question`, not the rewritten query. This stops a query rewrite that drifted from the intent from "validating" irrelevant chunks. Grading is one LLM call per chunk, run concurrently (up to 5 at a time). Tradeoff: extra cost and latency per query (k=5 means 5 grader calls); batching all chunks into one call would be cheaper but less reliable per chunk.

Termination. The retry loop is bounded by `MAX_RETRIES` (default 2), tracked in state. After that the system returns an honest fallback instead of generating from weak context.

Citations. The generator is told to cite `[n]`; the API only returns sources that were actually cited (or all relevant chunks if the model forgot to cite).

Temperatures. `0` for analysis, grading and rewriting-style routing decisions to keep them deterministic; a small value for generation.

Blocking endpoints. The graph, Chroma and SQLite calls are blocking, so the endpoints are plain `def` and FastAPI runs them in its threadpool. `/ingest` does the same via `run_in_threadpool`.

Assumptions

- English-language documentation; an OpenAI API key is available.
- A small corpus (3-5 documents), so no sharding, auth or rate limiting.
- Uploaded files are UTF-8 text (Markdown, text or HTML); PDFs are out of scope.
- `/feedback` is tied to a `query_id` returned by `/query`.

What I would improve with more time

- **Hybrid retrieval** (BM25 + vectors) and a **reranker**; exact API names and error strings are where pure embeddings are weakest.
- **Hallucination check node** (Self-RAG style) after generation that verifies each claim is supported by the context, regenerating or flagging if not.
- **Web-search fallback** (Tavily) before giving up, and **conversation memory** using a LangGraph checkpointer keyed by `session_id`.
- A proper **evaluation set** (e.g. RAGAS: faithfulness, answer relevance, context precision) to tune chunk size, k and the grader prompt with numbers instead of intuition.
- **Streaming** responses, **auth and rate limiting**, structured request tracing, and a **Dockerfile**.
- Batch or cache the relevance grading to cut cost and latency.